Raspberry Pi 5 AutoCAR

Raspberry Pi 5 電腦視覺自走車專案，整合網頁車控、相機錄影、YOLO26 視覺推論與自動駕駛。

目錄

- 專案文件與部署紀錄
- 專案功能
- 快速部署
- YOLO26 畫框預覽模型
- 硬體控制設定
- 手動車控
- 開發驗證

專案文件與部署紀錄

板端驗證紀錄：**2026-09-16**。原測試機：`rpi5@192.168.0.160`，控制頁面： http://192.168.0.160:8000。使用者已確認新版可正常操作。 本機程式與此次部署包一致，板端前端檔案已比對雜湊；詳見 部署備忘錄第 26 節。

`RPI5_AutoCAR-YOLO-integrated.zip` 已更新為本次程式、測試、設定範例與文件； 不含模型、虛擬環境與錄影。

重新打包：

```
python scripts/package_release.py
```

完整的從零開始安裝、模型匯出、硬體設定、服務管理與故障排除，請參閱 Raspberry Pi 5 AutoCAR + YOLO26 完整開發與部署備忘錄。

若要依照實際製作順序逐章操作，請使用 Raspberry Pi 5 AutoCAR + YOLO26 從零實作教學手冊。

道路影片抽幀、Roboflow 多邊形標記、反光增強與 YOLO26 訓練，請參閱 道路分割與反光改善實作課程。

專案功能

Raspberry Pi 5 電腦視覺自走車的基礎架構，包含：

- 手機 / 電腦瀏覽器 Web UI
- 啟用後四輪定速前進，F/B 逐步加減速、L/R 逐步差速轉向與立即停止
- 啟用 / 停用車體控制
- Raspberry Pi Camera 開始 / 停止 H.264 錄影
- Web UI 內建 YOLO26 NCNN 畫框 MJPEG 即時預覽，可與車控及錄影同時使用
- 網頁可獨立啟停視覺推論；關閉時顯示原始畫面供手控錄影與訓練資料取景
- 相機 180° 旋轉與雙軸伺服雲台控制（俯仰、迴旋、置中）
- 軟體 watchdog：控制訊號逾時自動停車

- PCA9685 四輪馬達與雙軸雲台控制
- YOLO26-seg 車道遮罩中心偵測（可選 OpenCV fallback）、anti-windup PID 與左右輪連續 PWM
- YOLO26 / NCNN 紅黃綠燈及停止標誌異步推論與 3/5 幀穩定判定
- Web UI 自動駕駛開關與即時遙測（原有手動功能完整保留）

快速部署

操作流程：Windows PowerShell → Raspberry Pi → Windows 瀏覽器。

以下將 `alice` 換成你的 **Pi 登入帳號**，`192.168.1.50` 換成 **Pi 的 IP**。 不需要修改 hostname。

開始前請確認：

- Pi 已啟用 SSH，並可從 Windows 連線。
- Windows PowerShell 可使用 `scp`、`ssh` 指令。

1. 在 Windows PowerShell 下載並上傳壓縮包

在你要存放壓縮包的資料夾開啟 PowerShell，執行：

```
Invoke-WebRequest -Uri
"https://raw.githubusercontent.com/wangsungwen/PI5_AUTOCAR/main/RPI5_AutoCAR-YOLO-
integrated.zip" -OutFile ".\RPI5_AutoCAR-YOLO-integrated.zip"

scp .\RPI5_AutoCAR-YOLO-integrated.zip alice@192.168.1.50:~/
```

`git clone` 用來複製 Git 倉庫，不能下載 `/blob/main/...zip` 的單一檔案； 這裡使用原始檔案網址直接下載 ZIP。若目前資料夾已經有最新版壓縮包，可直接執行 `scp`。

輸入 Pi 帳號密碼。首次連線確認主機指紋無誤後，輸入 `yes`。

2. 從 PowerShell 登入 Pi

```
ssh alice@192.168.1.50
```

> 以下第 3 ~ 5 步的指令都在 **登入後的 Pi 終端機**執行。

3. 解壓縮

```
sudo apt-get update
sudo apt-get install -y unzip

mkdir -p ~/RPI5_AutoCAR
unzip -o ~/RPI5_AutoCAR-YOLO-integrated.zip -d ~/RPI5_AutoCAR

cd ~/RPI5_AutoCAR
```

`-o` 會覆蓋解壓目錄內的同名檔案。

4. 執行一鍵安裝

```
sudo bash scripts/install.sh
```

安裝器會自動使用目前登入帳號，安裝相依套件、建立服務、設定開機啟動並啟動 AutoCAR。 首次安裝需要下載 AI 套件，請等到完成。

5. 確認 systemd 服務

```
sudo systemctl status autocar --no-pager
```

正常應顯示 `active (running)`。若要手動重新載入並確保開機啟動：

```
sudo systemctl daemon-reload
sudo systemctl enable --now autocar
```

修改 `/etc/autocar.env` 後，執行：

```
sudo systemctl restart autocar
```

查看錯誤紀錄：

```
sudo journalctl -u autocar -n 100 --no-pager
```

6. 在 Windows 瀏覽器開啟

```
http://192.168.1.50:8000
```

請使用自己的 Pi IP；若已修改 `AUTOCAR_PORT`，請使用設定的連接埠。

壓縮包不含模型，視覺辨識與自動駕駛所需模型須另外部署。 舊版升級若換了帳號，請檢查 `/etc/autocar.env` 的 `AUTOCAR_RECORDINGS`， 避免仍指向 `/home/rpi5`。移除或註解該設定，可使用新服務帳號家目錄下的 `Videos/autocar`； 修改後須重新啟動服務。

指定服務帳號與其他升級細節請見通用安裝說明。

YOLO26 畫框預覽模型

模型目錄與基本設定

將完整 NCNN 模型目錄放在 `/opt/autocar/models/yolo26n_ncnn_model/`，其中應包含 `metadata.yaml`、`.param` 與 `.bin`。預覽模型、固定輸入尺寸、信心門檻及相機編號由 `/etc/autocar.env` 控制：

```
AUTOCAR_CAMERA_INDEX=0
AUTOCAR_CAMERA_ROTATE_180=1
AUTOCAR_PREVIEW_MODEL=models/yolo26n_ncnn_model
AUTOCAR_PREVIEW_IMGSZ=640
AUTOCAR_PREVIEW_CONFIDENCE=0.40
AUTOCAR_PREVIEW_MAX_DET=20
AUTOCAR_PREVIEW_JPEG_QUALITY=65
```

外接 Webcam 可將 `AUTOCAR_CAMERA_INDEX` 改為實際編號，通常是 `1`，並將 `AUTOCAR_CAMERA_ROTATE_180=0`。NCNN 的 `AUTOCAR_PREVIEW_IMGSZ` 必須與匯出尺寸相同。

模型尺寸與執行期切換

2026-09-10 板端最終驗證成功的預覽組合為：

```
AUTOCAR_PREVIEW_MODEL=models/yolo26n_ncnn_model
AUTOCAR_PREVIEW_IMGSZ=640
```

曾出錯的組合是 `yolo26n_320_ncnn_model` 配上 `AUTOCAR_PREVIEW_IMGSZ=640`。新版後端 會讀取模型的 `metadata.yaml`；若固定尺寸不一致，會回傳 `NCNN input mismatch`、保持 推論關閉並繼續顯示原始畫面。

網頁現已支援執行期切換推論模型與固定尺寸。先關閉「視覺推論」，在「推論模型與固定 尺寸」選擇已安裝的 640 或 320 模型，按「套用」後再開啟推論。選項由 `/opt/autocar/models/*/metadata.yaml` 自動建立，尺寸不可手動填寫，因此不會把 320 模型 誤配成 640。此切換只在本次服務執行期間有效；重新啟動服務後，以 `/etc/autocar.env` 的 `AUTOCAR_PREVIEW_MODEL` 與 `AUTOCAR_PREVIEW_IMGSZ` 為準。

Pi 5 服務會在接受第一個相機串流請求前先載入 PyTorch / NCNN。這個順序可避免 Picamera2 先載入另一組原生 BLAS 函式庫後，造成 `libtorch_cpu.so: undefined symbol: sbgemm_`。

匯出較小的模型

專案設定範例預設 NCNN 為 **640 × 640**。2026-09-16 板端執行中為 `models/yolo26n_320_ncnn_model` / **320 × 320**；這是狀態 API 的執行期觀察， 未讀取板端 `/etc/autocar.env`，不推定重啟後仍為同一組。若要提高 Pi 5 的 FPS，建議先從原始 `.pt` 重新匯出 320 × 320 版本，並同時設定模型路徑及尺寸：

```
cp yolo26n.pt yolo26n_320.pt
yolo export model=yolo26n_320.pt format=ncnn imgsz=320 batch=1 device=cpu
```

```
AUTOCAR_PREVIEW_MODEL=models/yolo26n_320_ncnn_model
AUTOCAR_PREVIEW_IMGSZ=320
```

128 × 128 技術上可用，但遠處交通燈、停止標誌及其他小物件很容易漏檢，只建議作效能 實驗。降低尺寸可減輕 CPU、熱量及記憶體負擔，但無法修復缺少 ncnn、相機被占用、 NumPy / PyTorch 衝突或 NCNN 模型尺寸不匹配。完整匯出、部署、FPS 測試與當機判斷流程 請見部署備忘錄第 10、19、21 節。

視覺推論開關與錯誤處理

網頁的「視覺推論」開關預設為關閉：此時 /api/camera/stream 提供不含偵測框的原始 MJPEG，手動車控及 H.264 錄影都可正常使用。開啟後才會載入 YOLO26 NCNN 並輸出畫框 串流。啟動自動駕駛時系統會自動開啟視覺推論；自動駕駛運行中不可單獨關閉推論。 開啟前，後端會依序載入 OpenCV、NumPy、PyTorch、Ultralytics，最後才載入 NCNN；若發現 libtorch_cpu.so: undefined symbol: sbgemm_ 等二進位相依錯誤，開關會回復關閉並繼續 顯示原始畫面，不會讓手控與取景功能一起失效。

若推論已進入工作執行緒後才發生錯誤，後端也會自動把推論狀態改為關閉，瀏覽器偵測到 狀態轉換後重新連接 /api/camera/stream，因此黑色的失效畫框串流會自動降級為原始畫面； 錯誤文字仍保留在「辨識」欄供維修判斷。

載入順序不可把 ncnn 放在 torch 前面。兩者都包含原生運算函式庫；本次排查中， 單獨 PyTorch、完整 AI runtime、Picamera2 加完整 runtime，以及 640 NCNN 單張推論均 成功。專案因此固定使用已驗證的 torch → ultralytics → ncnn 順序，並另外在進入 推論前檢查模型固定尺寸，避免把尺寸問題誤判為套件問題。

AI runtime 修復

遇到 sbgemm_ 時，先停止服務並重裝正式環境中的 NumPy / SciPy 與 CPU PyTorch：

```
sudo systemctl stop autocar.service
sudo /opt/autocar-venv/bin/python -m pip install --upgrade --force-reinstall --no-
cache-dir numpy scipy
sudo /opt/autocar-venv/bin/python -m pip install --upgrade --force-reinstall --no-
cache-dir torch torchvision --index-url https://download.pytorch.org/whl/cpu
sudo systemctl restart autocar.service
```

新版 scripts/install_pi.sh 會在啟動服務前執行聯合載入測試；第一次失敗時自動重裝上述 套件並再測一次，第二次仍失敗則停止安裝，不會帶著損壞的推論環境啟動服務。

Pi 5 上禁止執行 pip install --force-reinstall ultralytics ncnn。此命令會要求 pip 重裝 Ultralytics 的全部相依套件，可能把已裝好的 torch+cpu 換成 PyPI CUDA 版本，接著下載 CUDA Toolkit、cuDNN、cuBLAS、Triton 等數 GB 檔案，最後以 No space left on device 失敗。CPU PyTorch 安裝完成後，如只需修復 Ultralytics，應使用：

```
sudo /opt/autocar-venv/bin/python -m pip install --upgrade --no-cache-dir --no-
deps ultralytics
sudo /opt/autocar-venv/bin/python -m pip install --upgrade --no-cache-dir ncnn
```

硬體控制設定

本車使用 I²C PCA9685 四輪驅動板，設定如下：

```
AUTOCAR_GPIO=pca9685
AUTOCAR_MANUAL_SPEED=50
AUTOCAR_GIMBAL=pca9685
AUTOCAR_I2C_BUS=1
AUTOCAR_PCA9685_ADDRESS=0x40
AUTOCAR_PAN_CHANNEL=12
AUTOCAR_TILT_CHANNEL=13
AUTOCAR_PAN_CENTER=0
AUTOCAR_TILT_CENTER=-80
AUTOCAR_TILT_INVERT=1
AUTOCAR_TILT_LOGICAL_MIN=-40
AUTOCAR_TILT_LOGICAL_MAX=10
```

馬達 A/B/C/D 的 PWM 通道分別為 0/5/6/11；方向通道依教材 `motor_control.py`，馬達 D 方向使用 BCM25/24。修改設定後：

```
sudo systemctl restart autocar
sudo journalctl -u autocar -f
```

部署前可用 `sudo i2cdetect -y 1` 確認 `0x40` 裝置存在。

新安裝的錄影儲存在服務帳號家目錄下的 `Videos/autocar`；既有 `AUTOCAR_RECORDINGS` 設定優先。服務狀態：

```
systemctl status autocar
```

自動駕駛的模型訓練、部署、參數與安全調校請見 開發手冊。手動模式在沒有模型時仍可使用；預設安全設定會阻止缺少交通或車道模型時啟動自動駕駛。

手動車控

更新日期：**2026-09-16**。

服務啟動時車體仍為停用；在 UI 開啟「車體控制」後，四輪以 `AUTOCAR_MANUAL_SPEED`（預設 50% PWM）同速前進。四輪在同一控制週期更新， 取消舊版左右兩側之間的 10 ms 延遲；I2C 寫入仍依序執行。

操作	行為
F / ↑ / W	每次提高轉速設定 5%，最高 100%
B / ↓ / S	每次降低轉速設定 5%，最低 0%，不倒車
L / ← / A	每次左調 5°，最低 −45°

操作	行為
R / → / D	每次右調 5°，最高 +45°
轉向歸零	恢復直行，維持轉速設定
STOP / 空白鍵	四輪停止，轉速與轉向歸零；F 可由 0% 再起步

每次點擊 / 按鍵只調整一次，長按不累加、放開維持設定。相反方向的轉向按鍵 會先逐步回正，再往另一側增加角度。UI 顯示速度設定、轉向設定及 A/B/C/D 各輪 PWM。沒有編碼器與角度感測器，這些不是實測 RPM 或車頭轉過的角度。

沿用參考程式接線：A/C 左側，B/D 右側。轉向採內側輪降速， `內側 PWM = 速度設定 × (1 − 0.8 × |轉向設定| / 最大角度)`，外側輪保持速度設定。 因此加減速會同步調整四輪輸出，轉彎時左右輸出不同；最大轉向時內側保留 20%。

可在環境設定調整 `AUTOCAR_MANUAL_SPEED_STEP=5`、 `AUTOCAR_MANUAL_STEERING_STEP=5`、 `AUTOCAR_MANUAL_STEERING_MAX=45`。 瀏覽器每 200 ms 呼叫 `/api/vehicle/heartbeat`，不重送加減速指令；控制中斷超過 `AUTOCAR_WATCHDOG_SECONDS`（預設 0.8 秒）會停止，心跳恢復也不會自動起步。 切換視窗或隱藏頁面會送 STOP。自動駕駛保留獨立連續左右輪控制，啟用時不觸發 手動定速前進，手動心跳不延長自駕 watchdog。

開發驗證

在專案根目錄執行：

```
python -m unittest discover -s tests -v
node tests/test_manual_ui.js
```